

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №3

Вычислительная математика

Вариант 8

Выполнил:
Караганов Павел

Группа: Р3210

Проверил:
Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

1 Цель лабораторной работы

Найти приближённое значение определённого интеграла с требуемой точностью различными численными методами, применить правило Рунге для оценки погрешности и рассмотреть несобственные интегралы.

2 Порядок выполнения работы

Работа состоит из двух частей: программной и вычислительной.

Программная часть:

1. Разработать программу для численного интегрирования функций.
2. Реализовать методы: левых, правых, средних прямоугольников, трапеций и Симпсона.
3. Добавить применение правила Рунге для оценки погрешности.
4. Вывести результаты, требуемое число разбиений и оценку погрешности.

Вычислительная реализация задачи:

1. Вычислить интеграл, приведенный в таблице (Вариант 8), точно.
2. Вычислить интеграл по формуле Ньютона – Котеса при $n = 6$.
3. Вычислить интеграл по формулам средних прямоугольников, трапеций и Симпсона при $n = 10$.
4. Сравнить результаты с точным значением интеграла.
5. Определить относительную погрешность вычислений для каждого метода.
6. В отчете **отразить последовательные вычисления**.

Для варианта 8 задан интеграл:

$$\int_2^3 (3x^3 - 2x^2 - 7x - 8) dx$$

3 Рабочие формулы методов

3.1 Метод средних прямоугольников

$$I \approx h \sum_{i=0}^{n-1} f(x_{i+0.5}), \quad \text{где } x_{i+0.5} = a + (i + 0.5)h$$

Порядок точности: $O(h^2)$.

3.2 Метод трапеций

$$I \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right)$$

Порядок точности: $O(h^2)$.

3.3 Метод Симпсона

$$I \approx \frac{h}{3} \left(f(x_0) + 4 \sum_{i=1,3,5,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{n-2} f(x_i) + f(x_n) \right)$$

Порядок точности: $O(h^4)$.

3.4 Формула Ньютона-Котеса (замкнутого типа)

$$I \approx (b-a) \sum_{i=0}^n H_i f(x_i)$$

Где H_i — коэффициенты Котеса, зависящие от n .

4 Вычислительная реализация задачи (последовательные вычисления)

Функция: $f(x) = 3x^3 - 2x^2 - 7x - 8$, интервал интегрирования: $[2, 3]$.

4.1 Точное вычисление интеграла (Ньютон-Лейбниц)

Найдём первообразную:

$$F(x) = \int (3x^3 - 2x^2 - 7x - 8) dx = \frac{3x^4}{4} - \frac{2x^3}{3} - \frac{7x^2}{2} - 8x + C$$

Вычислим значения на концах отрезка:

$$F(3) = \frac{3 \cdot 81}{4} - \frac{2 \cdot 27}{3} - \frac{7 \cdot 9}{2} - 24 = 60.75 - 18 - 31.5 - 24 = -12.75$$

$$F(2) = \frac{3 \cdot 16}{4} - \frac{2 \cdot 8}{3} - \frac{7 \cdot 4}{2} - 16 = 12 - 5.333 - 14 - 16 = -23.333$$

Применим формулу Ньютона-Лейбница:

$$I_{\text{точно}} = F(3) - F(2) = -12.75 - (-23.333) = 10.583333...$$

Точное значение: $I_{\text{точно}} = 10 + \frac{7}{12} \approx 10.583333$.

4.2 Вычисление по формуле Ньютона-Котеса ($n = 6$)

Шаг интегрирования: $h = \frac{3-2}{6} = \frac{1}{6} \approx 0.16667$.

Узлы $x_i = a + ih$:

$$x_0 = 2.0; x_1 = \frac{13}{6}; x_2 = \frac{7}{3}; x_3 = \frac{5}{2}; x_4 = \frac{8}{3}; x_5 = \frac{17}{6}; x_6 = 3.0$$

Коэффициенты Котеса для $n = 6$:

$$H_0 = \frac{41}{840}, H_1 = \frac{216}{840}, H_2 = \frac{27}{840}, H_3 = \frac{272}{840},$$
$$H_4 = \frac{27}{840}, H_5 = \frac{216}{840}, H_6 = \frac{41}{840}$$

Подстановка в формулу $I \approx (b-a) \sum H_i f(x_i)$:

$$I_{\text{НК}} = 1 \cdot \frac{1}{840} \left(41f(2) + 216f\left(\frac{13}{6}\right) + 27f\left(\frac{7}{3}\right) + 272f(2.5) + \dots + 41f(3) \right)$$
$$= \frac{1}{840} (41(-6) + 216(-2.902) + 27(1.185) + 272(6.375) + \dots + 41(34))$$
$$= 10.583333$$

Поскольку $f(x)$ — полином 3-й степени, а метод Ньютона-Котеса при $n = 6$ точен для полиномов до 7-й степени, относительная погрешность равна 0.

4.3 Вычисление по формулам при $n = 10$

Шаг разбиения: $h = \frac{b-a}{n} = \frac{3-2}{10} = 0.1$.

Узлы: $x_0 = 2.0, x_1 = 2.1, x_2 = 2.2, \dots, x_9 = 2.9, x_{10} = 3.0$.

Средние прямоугольники: Промежуточные узлы: $x_{i+0.5} \in \{2.05, 2.15, 2.25, \dots, 2.95\}$.

$$\begin{aligned} I_{\text{ср.пр}} &= h \cdot (f(2.05) + f(2.15) + f(2.25) + \dots + f(2.95)) \\ &= 0.1 \cdot (-4.606 - 2.999 - 1.164 + \dots + 31.541) \\ &= 10.56625 \end{aligned}$$

Трапеции:

$$\begin{aligned} I_{\text{трап}} &= h \cdot \left(\frac{f(2.0) + f(3.0)}{2} + f(2.1) + f(2.2) + \dots + f(2.9) \right) \\ &= 0.1 \cdot \left(\frac{-6 + 34}{2} - 4.517 - 2.816 + \dots + 29.177 \right) \\ &= 10.6175 \end{aligned}$$

Симпсон:

$$\begin{aligned} I_{\text{симп}} &= \frac{h}{3} \cdot (f(2.0) + 4[f(2.1) + f(2.3) + \dots] + 2[f(2.2) + f(2.4) + \dots] + f(3.0)) \\ &= \frac{0.1}{3} \cdot (-6 + 4(31.432) + 2(24.312) + 34) \\ &= 10.58333 \end{aligned}$$

4.4 Сравнение результатов и определение относительной погрешности

Относительная погрешность рассчитывается по формуле: $\delta = |I_{\text{вычисл}} - I_{\text{точно}}| / |I_{\text{точно}}| \cdot 100\%$.

Метод	n	Значение	Относительная погрешность (δ)
Точное значение	—	10.58333	—
Ньютона-Котеса	6	10.58333	0%
Средние прямоугольники	10	10.56625	0.161%
Трапеции	10	10.61750	0.322%
Симпсон	10	10.58333	0%

Анализ вычислений:

1. Метод Ньютона-Котеса и метод Симпсона выдали точный результат ($\delta = 0\%$). Это математически закономерно, так как подынтегральная функция является полиномом третьей степени, а оба эти метода обеспечивают безошибочное интегрирование полиномов степени 3 и выше.
2. Метод средних прямоугольников оказался в 2 раза точнее метода трапеций (0.161% против 0.322%), что наглядно подтверждает теоретическое правило для гладких функций: главная

часть погрешности метода средних прямоугольников вдвое меньше погрешности метода трапеций (с обратным знаком).

5 Вывод

В ходе лабораторной работы были реализованы и применены методы численного интегрирования: Ньютона-Котеса, средних прямоугольников, трапеций и Симпсона. Для интеграла

$$\int_2^3 (3x^3 - 2x^2 - 7x - 8) dx$$

получено точное значение $I = 10.58333$ и выполнено сравнение с результатами численных методов.

Сравнение показало, что методы Ньютона-Котеса ($n = 6$) и Симпсона ($n = 10$) дали значение, совпадающее с точным (относительная погрешность 0%), что соответствует теории для полинома третьей степени. Метод средних прямоугольников дал меньшую погрешность (0.161%), чем метод трапеций (0.322%), то есть оказался более точным на выбранной сетке.

Таким образом, цель работы достигнута: изучены и проверены на практике основные квадратурные формулы, выполнена оценка точности результатов и подтверждены теоретические свойства методов численного интегрирования.

6 Листинг программы

```
import math
from dataclasses import dataclass
from typing import Callable

Function = Callable[[float], float]

@dataclass
class ImproperFunction:
    name: str
    func: Function
    a: float
    b: float
    singular_points: tuple[float, ...]

MAX_ABS_VALUE = 1e10
MAX_SUBDIVISIONS = 1_000_000

def ensure_finite_and_bounded(value: float, context: str) -> float:
    if not math.isfinite(value) or abs(value) > MAX_ABS_VALUE:
        raise OverflowError(f"Слишком большое число или переполнение при {context}.")
    return value

def ensure_valid_subdivisions(n: int, context: str) -> None:
    if n <= 0:
        raise ValueError(f"Число разбиений n должно быть положительным при {context}.")
    if n > MAX_SUBDIVISIONS:
```

```

        raise ValueError(
            f"Число разбиений n={n} превышает допустимый предел {MAX_SUBDIVISIONS} при {context}."
        )

def evaluate_function(func: Function, x: float) -> float:
    y = float(func(x))
    return ensure_finite_and_bounded(y, f"вычислении f(x) (x={x:.6g})")

def rectangle_left(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "методе левых прямоугольников")
    h = (b - a) / n
    ensure_finite_and_bounded(h, "вычислении шага h")

    acc = 0.0
    for i in range(n):
        acc += evaluate_function(func, a + i * h)
        ensure_finite_and_bounded(acc, "накоплении суммы (левые прямоугольники)")

    return ensure_finite_and_bounded(h * acc, "вычислении интеграла (левые прямоугольники)")

def rectangle_right(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "методе правых прямоугольников")
    h = (b - a) / n
    ensure_finite_and_bounded(h, "вычислении шага h")

    acc = 0.0
    for i in range(1, n + 1):
        acc += evaluate_function(func, a + i * h)
        ensure_finite_and_bounded(acc, "накоплении суммы (правые прямоугольники)")

    return ensure_finite_and_bounded(h * acc, "вычислении интеграла (правые прямоугольники)")

def rectangle_midpoint(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "методе средних прямоугольников")
    h = (b - a) / n
    ensure_finite_and_bounded(h, "вычислении шага h")

    acc = 0.0
    for i in range(n):
        acc += evaluate_function(func, a + (i + 0.5) * h)
        ensure_finite_and_bounded(acc, "накоплении суммы (средние прямоугольники)")

    return ensure_finite_and_bounded(h * acc, "вычислении интеграла (средние прямоугольники)")

def trapezoid(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "методе трапеций")
    h = (b - a) / n
    ensure_finite_and_bounded(h, "вычислении шага h")

```

```

s = 0.5 * (evaluate_function(func, a) + evaluate_function(func, b))
ensure_finite_and_bounded(s, "начальной сумме (трапеции)")

for i in range(1, n):
    s += evaluate_function(func, a + i * h)
    ensure_finite_and_bounded(s, "накоплении суммы (трапеции)")

return ensure_finite_and_bounded(h * s, "вычислении интеграла (трапеции)")

def simpson(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "методе Симпсона")
    if n % 2 != 0:
        n += 1
        ensure_valid_subdivisions(n, "коррекции n до четного в методе Симпсона")

    h = (b - a) / n
    ensure_finite_and_bounded(h, "вычислении шага h")

    odd_sum = 0.0
    for i in range(1, n, 2):
        odd_sum += evaluate_function(func, a + i * h)
        ensure_finite_and_bounded(odd_sum, "накоплении нечетной суммы (Симпсон)")

    even_sum = 0.0
    for i in range(2, n, 2):
        even_sum += evaluate_function(func, a + i * h)
        ensure_finite_and_bounded(even_sum, "накоплении четной суммы (Симпсон)")

    result = (h / 3.0) * (
        evaluate_function(func, a)
        + evaluate_function(func, b)
        + 4.0 * odd_sum
        + 2.0 * even_sum
    )
    return ensure_finite_and_bounded(result, "вычислении интеграла (Симпсон)")

def solve_linear_system(a: list[list[float]], b: list[float]) -> list[float]:
    n = len(a)
    aug = [row[:] + [b_val] for row, b_val in zip(a, b)]

    for col in range(n):
        pivot = max(range(col, n), key=lambda r: abs(aug[r][col]))
        if abs(aug[pivot][col]) < 1e-14:
            raise ValueError("Система вырождена, коэффициенты Ньютона-Котеса не вычислены")
        aug[col], aug[pivot] = aug[pivot], aug[col]

        div = aug[col][col]
        for j in range(col, n + 1):
            aug[col][j] /= div

    for i in range(n):
        if i == col:
            continue

```

```

        factor = aug[i][col]
        for j in range(col, n + 1):
            aug[i][j] -= factor * aug[col][j]

    return [aug[i][n] for i in range(n)]

def newton_cotes_closed(func: Function, a: float, b: float, n: int) -> float:
    ensure_valid_subdivisions(n, "формуле Ньютона-Котеса")
    matrix = []
    rhs = []
    for k in range(n + 1):
        matrix.append([float(i**k) for i in range(n + 1)])
        rhs.append((n ** (k + 1)) / float(k + 1))

    c = solve_linear_system(matrix, rhs)
    h = (b - a) / n
    weighted_sum = sum(c[i] * evaluate_function(func, a + i * h) for i in range(n + 1))
    return h * weighted_sum

def integrate_with_runge(
    method: Callable[[Function, float, float, int], float],
    p: int,
    func: Function,
    a: float,
    b: float,
    eps: float,
    n0: int = 4,
    max_iter: int = 30,
    improper: bool = False,
) -> tuple[float, int, float]:
    if improper:
        max_iter = min(max_iter, 15)

    ensure_valid_subdivisions(n0, "начальном числе разбиений по Рунге")
    n = n0
    i_h = method(func, a, b, n)
    ensure_finite_and_bounded(i_h, "начальном приближении по Рунге")

    for _ in range(max_iter):
        if n > MAX_SUBDIVISIONS // 2:
            raise RuntimeError(
                f"Превышен допустимый предел разбиений n>{MAX_SUBDIVISIONS} в правиле Рунге"
            )
        n2 = n * 2
        ensure_valid_subdivisions(n2, "удвоении числа разбиений по Рунге")
        i_h2 = method(func, a, b, n2)
        ensure_finite_and_bounded(i_h2, "уточненном приближении по Рунге")
        runge_error = abs(i_h2 - i_h) / (2**p - 1)
        ensure_finite_and_bounded(runge_error, "оценке погрешности по Рунге")

        if runge_error < eps:
            return i_h2, n2, runge_error

    n = n2

```



```

        i_h = i_h2

        raise RuntimeError("Не удалось достичь требуемой точности за разумное число
итераций")

def antiderivative_variant_8(x: float) -> float:
    return 0.75 * x**4 - (2.0 / 3.0) * x**3 - 3.5 * x**2 - 8.0 * x

def f_variant_8(x: float) -> float:
    return 3.0 * x**3 - 2.0 * x**2 - 7.0 * x - 8.0

def exact_integral_variant_8(a: float = 2.0, b: float = 3.0) -> float:
    return antiderivative_variant_8(b) - antiderivative_variant_8(a)

def relative_error(approx: float, exact: float) -> float:
    if abs(exact) < 1e-14:
        return abs(approx - exact)
    return abs((approx - exact) / exact)

def build_truncated_intervals(
    a: float,
    b: float,
    singular_points: tuple[float, ...],
    delta: float,
) -> list[tuple[float, float]]:
    inner_singulars = sorted(s for s in singular_points if a < s < b)
    all_points = [a] + inner_singulars + [b]
    intervals: list[tuple[float, float]] = []

    singular_set = set(singular_points)
    for i in range(len(all_points) - 1):
        left = all_points[i]
        right = all_points[i + 1]

        if left in singular_set:
            left += delta
        if right in singular_set:
            right -= delta

        if right <= left:
            continue
        intervals.append((left, right))

    return intervals

def approximate_improper_integral(
    func: Function,
    a: float,
    b: float,
    singular_points: tuple[float, ...],
    method: Callable[[Function, float, float, int], float],

```

```

    p: int,
    eps: float,
    max_outer: int = 10,
) -> tuple[bool, float | None]:
    length = b - a
    delta = length / 8.0
    prev: float | None = None
    divergence_count = 0

    for iteration in range(max_outer):
        intervals = build_truncated_intervals(a, b, singular_points, delta)
        if not intervals:
            return False, None

        total = 0.0
        try:
            for left, right in intervals:
                part, _, _ = integrate_with_runge(
                    method, p, func, left, right, eps / 10.0, n0=4, improper=True
                )
                total += part
        except (ValueError, RuntimeError, ZeroDivisionError, OverflowError):
            return False, None

        if not math.isfinite(total) or abs(total) > 1e12:
            return False, None

        if prev is not None:
            if abs(total) > abs(prev) * 1.5:
                divergence_count += 1
            else:
                divergence_count = 0

            if divergence_count >= 2:
                return False, None

            if abs(total - prev) < eps:
                return True, total

        prev = total
        delta /= 2.0

        if delta < 1e-12:
            break

    return False, None

def print_computational_part_variant_8() -> None:
    a, b = 2.0, 3.0
    exact = exact_integral_variant_8(a, b)

    nc_n6 = newton_cotes_closed(f_variant_8, a, b, 6)
    mid_n10 = rectangle_midpoint(f_variant_8, a, b, 10)
    trap_n10 = trapezoid(f_variant_8, a, b, 10)
    simp_n10 = simpson(f_variant_8, a, b, 10)

```

```

print("\n=== Вычислительная часть ===")
print("Интеграл:  $\int[2,3] (3x^3 - 2x^2 - 7x - 8) dx$ ")
print(f"Точное значение (Ньютон-Лейбниц): {exact:.10f}")
print(f"Ньютона-Котеса (n=6): {nc_n6:.10f}")
print(f"Средние прямоугольники (n=10): {mid_n10:.10f}")
print(f"Трапеции (n=10): {trap_n10:.10f}")
print(f"Симпсон (n=10): {simp_n10:.10f}")

print("\nОтносительные погрешности:")
print(f"Ньютона-Котеса (n=6): {relative_error(nc_n6, exact):.10e}")
print(f"Средние прямоугольники (n=10): {relative_error(mid_n10, exact):.10e}")
print(f"Трапеции (n=10): {relative_error(trap_n10, exact):.10e}")
print(f"Симпсон (n=10): {relative_error(simp_n10, exact):.10e}")

def normalize_number_input(raw: str) -> str:
    return (
        raw.strip()
        .replace("\u00a0", "")
        .replace(" ", "")
        .replace(",", ".")
        .replace("_", "-")
        .replace("-", "-")
    )

def read_float(prompt: str, max_decimals: int | None = None) -> float:
    while True:
        raw = normalize_number_input(input(prompt))
        try:
            if not raw:
                raise ValueError

            if max_decimals is not None:
                mantissa = raw.split("e", 1)[0].split("E", 1)[0]
                if "." in mantissa:
                    decimals = len(mantissa.split(".", 1)[1])
                    if decimals > max_decimals:
                        raise ValueError(
                            f"Допускается не более {max_decimals} знаков после запятой."
                        )

            value = float(raw)
            return ensure_finite_and_bounded(value, "вводе числа")
        except ValueError as exc:
            if exc.args and isinstance(exc.args[0], str) and exc.args[0]:
                print(f"Ошибка ввода. {exc.args[0]}")
            else:
                print("Ошибка ввода. Введите число, например 0,01 (или 0.01)")
        except OverflowError as exc:
            print(f"Ошибка ввода: {exc}")

def read_int(prompt: str, min_value: int, max_value: int) -> int:
    while True:
        raw = input(prompt).strip()

```

```

        if raw.isdigit():
            value = int(raw)
            if min_value <= value <= max_value:
                return value
        print(f"Ошибка ввода. Введите целое число от {min_value} до {max_value}.")

def validate_domain_for_mandatory_function(function_choice: int, a: float, b: float) ->
None:
    left = min(a, b)
    right = max(a, b)

    if function_choice == 5 and left <= -2.0:
        left_str = f"{left:.6g}".replace(".", ",")
        right_str = f"{right:.6g}".replace(".", ",")
        raise ValueError(
            "Функция  $\ln(x + 2)$  определена только при  $x > -2$ . "
            f"Введенный отрезок [{left_str}, {right_str}] выходит за область
определения."
        )

def run_mandatory_assignment(
    functions: dict[int, tuple[str, Function]],
    methods: dict[int, tuple[str, Callable[[Function, float, float, int], float],
int]],
) -> None:
    print("\nДоступные функции:")
    for idx, (name, _) in functions.items():
        print(f"{idx}. f(x) = {name}")

    f_choice = read_int("Выберите функцию (1-5): ", 1, 5)
    _, func = functions[f_choice]

    a = read_float("Введите нижний предел интегрирования a: ")
    b = read_float("Введите верхний предел интегрирования b: ")

    if abs(b - a) < 1e-14:
        print("Интеграл на нулевом интервале равен 0.")
        return

    try:
        validate_domain_for_mandatory_function(f_choice, a, b)
    except ValueError as exc:
        print(f"Ошибка ввода: {exc}")
        return

    eps = read_float("Введите требуемую точность eps (>0): ", max_decimals=10)
    if eps <= 0:
        print("Точность должна быть положительной.")
        return

    print("\nМетоды:")
    for idx, (name, _, _) in methods.items():
        print(f"{idx}. {name}")

    m_choice = read_int("Выберите метод (1-5): ", 1, 5)

```

```

method_name, method_func, p = methods[m_choice]

try:
    value, n_final, runge_est = integrate_with_runge(method_func, p, func, a, b,
eps, n0=4)
except ValueError as exc:
    print(f"Ошибка вычисления: {exc}")
    return
except OverflowError as exc:
    print(f"Ошибка вычисления: {exc}")
    return
except RuntimeError as exc:
    print(f"Ошибка вычисления: {exc}")
    return

print("\n== Результат обязательного задания ==")
print(f"Метод: {method_name}")
print(f"Приближенное значение интеграла: {value:.10f}")
print(f"Число разбиений n: {n_final}")
print(f"Оценка погрешности по Рунге: {runge_est:.10e}")

print_computational_part_variant_8()

def run_optional_assignment(
    methods: dict[int, tuple[str, Callable[[Function, float, float, int], float],
int]],
) -> None:
    improper_functions: dict[int, ImproperFunction] = {
        1: ImproperFunction(
            name="f(x)=1/sqrt(x), [0,1], разрыв в точке a=0 (сходится)",
            func=lambda x: 1.0 / math.sqrt(x),
            a=0.0,
            b=1.0,
            singular_points=(0.0,)),
        2: ImproperFunction(
            name="f(x)=1/(x-1)^2, [0,2], разрыв внутри отрезка x=1 (расходится)",
            func=lambda x: 1.0 / ((x - 1.0) ** 2),
            a=0.0,
            b=2.0,
            singular_points=(1.0,)),
    }

    print("\nНеобязательное задание (2 функции):")
    for idx, item in improper_functions.items():
        print(f"{idx}. {item.name}")

    f_choice = read_int("Выберите функцию (1-2): ", 1, 2)
    selected = improper_functions[f_choice]
    name = selected.name
    func = selected.func
    a = selected.a
    b = selected.b
    singular_points = selected.singular_points

```

```

print(f"\nВыбрано: {name}")
print(f"Пределы интегрирования: [{a}, {b}]")

eps = read_float("Введите требуемую точность eps (>0): ", max_decimals=10)
if eps <= 0:
    print("Точность должна быть положительной.")
    return

print("\nМетоды для вычисления (при сходимости):")
for idx, (method_name, _, _) in methods.items():
    print(f"{idx}. {method_name}")

m_choice = read_int("Выберите метод (1-5): ", 1, 5)
method_name, method_func, p = methods[m_choice]

convergent, value = approximate_improper_integral(
    func, a, b, singular_points, method_func, p, eps
)

print("\n== Результат необязательного задания ==")
if not convergent or value is None:
    print("Интеграл не существует")
    return

print("Интеграл сходится")
print(f"Метод: {method_name}")
print(f"Приближенное значение несобственного интеграла: {value:.10f}")

def main() -> None:
    functions: dict[int, tuple[str, Function]] = {
        1: ("3x^3 - 2x^2 - 7x - 8", f_variant_8),
        2: ("sin(x)", lambda x: math.sin(x)),
        3: ("x^2 + 2x + 1", lambda x: x**2 + 2 * x + 1),
        4: ("exp(-x^2)", lambda x: math.exp(-(x**2))),
        5: ("ln(x + 2)", lambda x: math.log(x + 2.0)),
    }

    methods: dict[int, tuple[str, Callable[[Function, float, float, int], float], int]]
= {
        1: ("Левые прямоугольники", rectangle_left, 2),
        2: ("Правые прямоугольники", rectangle_right, 2),
        3: ("Средние прямоугольники", rectangle_midpoint, 2),
        4: ("Трапеции", trapezoid, 2),
        5: ("Симпсона", simpson, 4),
    }

    print("ЛР №3: Численное интегрирование")
    print("Обязательная программная часть: методы Ньютона-Котеса + правило Рунге")
    print("Необязательная часть: несобственные интегралы 2-го рода (2 функции)")

    while True:
        print("\n=== Главное меню ===")
        print("1. Обязательное задание")
        print("2. Необязательное задание")
        print("0. Выход")
        menu_choice = read_int("Выберите пункт (0-2): ", 0, 2)

```

```
if menu_choice == 0:
    print("Завершение работы.")
    break

if menu_choice == 1:
    run_mandatory_assignment(functions, methods)
else:
    run_optional_assignment(methods)

if __name__ == "__main__":
    main()
```